

GUÍA DE ESTUDIO COMPLETA — VELOUR KENO

Spring Boot · HTML · CSS · JavaScript

Objetivo: Que cada integrante de 4bytes pueda explicar al profesor QUÉ hace cada archivo, POR QUÉ se implementó así y CÓMO modificarlo si lo pide (ej: "hazme esta ventana azul", "hazme un login en Spring Boot").

PARTE 1 — SPRING BOOT (Backend)

1.1 ¿Qué es Spring Boot y cómo arranca?

Spring Boot es un framework de Java que simplifica la creación de aplicaciones web. En vez de configurar manualmente un servidor, Spring Boot lo hace automáticamente.

Archivo: `KenoApplication.java`

```
@Slf4j
@SpringBootApplication
public class KenoApplication {
    public static void main(String[] args) {
        SpringApplication.run(KenoApplication.class, args);
    }
}
```

Conceptos clave que el profesor puede preguntar:

- `@SpringBootApplication` — Es una anotación compuesta que combina tres:
 - `@Configuration` → Esta clase puede definir beans (objetos manejados por Spring)
 - `@EnableAutoConfiguration` → Spring configura automáticamente todo según las dependencias del pom.xml
 - `@ComponentScan` → Spring escanea automáticamente todos los paquetes hijos de `com.velour.keno` buscando clases anotadas con `@Service`, `@Controller`, `@Repository`, `@Component`
- `SpringApplication.run()` → Arranca el servidor embebido (Tomcat), carga toda la configuración y deja la app escuchando en el puerto 8080
- `@Slf4j` → Anotación de Lombok que auto-genera un logger. Permite usar `log.info("mensaje")` sin declararlo manualmente

Si el profesor pregunta "¿Cómo cambio el puerto?": → En `application.properties: server.port=9090`

1.2 Configuración — `application.properties`

```
server.port=8080
spring.datasource.url=jdbc:postgresql://localhost:5432/velour_keno
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
jwt.secret=velour-keno-secret-key-super-segura-2024-desarrollo
jwt.expiration=86400000
```

Cada línea explicada:

Propiedad	Qué hace	Si el profesor pregunta...
<code>server.port</code>	Puerto donde escucha la app	"Cámbialo al 3000" → <code>server.port=3000</code>
<code>spring.datasource.url</code>	URL de conexión JDBC a PostgreSQL	"Conecta a MySQL" → <code>jdbc:mysql://localhost:3306/keno</code>
<code>spring.datasource.username/password</code>	Credenciales de la BD	Solo cambiar los valores
<code>spring.jpa.hibernate.ddl-auto=update</code>	Hibernate actualiza las tablas automáticamente sin borrar datos	<code>create</code> = borra y recrea; <code>validate</code> = solo verifica
<code>spring.jpa.show-sql=true</code>	Muestra las queries SQL en la consola	Útil en desarrollo, en producción se pone <code>false</code>
<code>jwt.secret</code>	Clave secreta para firmar tokens JWT	Debe ser larga (+32 caracteres) y secreta
<code>jwt.expiration</code>	Duración del token en milisegundos	86400000 ms = 24 horas

Perfiles de Spring:

- `application.properties` → configuración local (desarrollo)
- `application-prod.properties` → producción (Render), usa variables de entorno: `${DATABASE_URL}`
- `application-test.properties` → tests con BD temporal (`ddl-auto=create-drop`)
- Se activan con: `spring.profiles.active=prod`

1.3 Dependencias — `pom.xml`

El `pom.xml` es el archivo de Maven que declara todas las librerías del proyecto.

Dependencias principales y para qué sirve cada una:

Dependencia	Qué aporta	Ejemplo en el proyecto
<code>spring-boot-starter-web</code>	Servidor web (Tomcat) + REST controllers	<code>@RestController</code> , <code>@GetMapping</code> , <code>@PostMapping</code>
<code>spring-boot-starter-security</code>	Autenticación, autorización, filtros	<code>SecurityConfig.java</code> , protege endpoints
<code>spring-boot-starter-data-jpa</code>	ORM con Hibernate, repositorios automáticos	<code>UserRepository</code> , <code>@Entity</code> , <code>@Table</code>
<code>spring-boot-starter-validation</code>	Validación de DTOs con anotaciones	<code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> , <code>@Pattern</code>
<code>spring-boot-starter-websocket</code>	WebSocket + STOMP para tiempo real	<code>WebSocketConfig.java</code> , chat, sorteos
<code>postgresql</code>	Driver JDBC para PostgreSQL	Conexión a la BD
<code>lombok</code>	Genera código automáticamente	<code>@Getter</code> , <code>@Setter</code> , <code>@Builder</code> , <code>@Slf4j</code>
<code>jjwt-api/impl/jackson</code>	Librería JWT (JSON Web Tokens)	<code>JwtTokenProvider.java</code> genera y valida tokens
<code>spring-boot-starter-test</code>	JUnit 5, Mockito para tests	<code>KenoApplicationTests.java</code>

Si el profesor pregunta "¿para qué es Lombok?": → Sin Lombok tendrías que escribir getters, setters, constructores, `toString`, etc. manualmente. Lombok genera todo eso con una sola anotación.

1.4 Entidades JPA — El modelo de datos

Las entidades son clases Java que representan tablas en la base de datos. JPA/Hibernate convierte automáticamente entre objetos Java y filas SQL.

Entidad `User.java`

```
@Entity
@Table(name = "users")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Column(unique = true, nullable = false, length = 50)
```

```
private String username;

>Email @NotBlank
@Column(unique = true, nullable = false, length = 100)
private String email;

@NotBlank
@Column(nullable = false)
private String password;

@Column(nullable = false, precision = 12, scale = 2)
@Builder.Default
private BigDecimal balance = new BigDecimal("1000.00");

@Column(nullable = false, length = 20)
@Builder.Default
private String role = "PLAYER";

@Column(nullable = false)
@Builder.Default
private Boolean active = true;

@Column(length = 20)
@Builder.Default
private String provider = "LOCAL";

@CreationTimestamp
@Column(updatable = false)
private LocalDateTime createdAt;

@UpdateTimestamp
private LocalDateTime updatedAt;
}
```

Anotaciones JPA explicadas una por una:

Anotación	Qué hace	Ejemplo
@Entity	Marca la clase como tabla en la BD	Sin esto, Hibernate la ignora
@Table(name = "users")	Nombre de la tabla en SQL	Si no se pone, usa el nombre de la clase
@Id	Clave primaria	Siempre necesaria en cada entidad
@GeneratedValue(strategy = IDENTITY)	Auto-incremento manejado por la BD	PostgreSQL usa SERIAL internamente
@Column(unique = true)	Restricción UNIQUE en la BD	No permite emails duplicados

Anotación	Qué hace	Ejemplo
<code>@Column(nullable = false)</code>	NOT NULL en la BD	El campo es obligatorio
<code>@Column(length = 50)</code>	VARCHAR(50)	Limita el tamaño del string
<code>@Column(precision = 12, scale = 2)</code>	DECIMAL(12,2)	Para dinero: 12 dígitos total, 2 decimales
<code>@NotBlank</code>	Validación: no puede estar vacío ni ser solo espacios	Error 400 si se viola
<code>@Email</code>	Validación: debe ser formato email válido	Usa regex internamente
<code>@Builder.Default</code>	Valor por defecto cuando se usa el patrón Builder	<code>balance = 1000.00</code> al registrarse
<code>@CreationTimestamp</code>	Hibernate pone la fecha automáticamente al crear	No se puede modificar después
<code>@UpdateTimestamp</code>	Hibernate actualiza la fecha al modificar	Se actualiza en cada <code>save()</code>

Anotaciones de Lombok:

- `@Getter` → Genera todos los `getUsername()`, `getEmail()`, etc.
- `@Setter` → Genera todos los `setUsername()`, `setEmail()`, etc.
- `@NoArgsConstructor` → Constructor sin parámetros (requerido por JPA)
- `@AllArgsConstructor` → Constructor con todos los campos
- `@Builder` → Patrón Builder: `User.builder().username("juan").email("j@e.com").build()`

Entidad `Game.java`

```
@Entity
@Table(name = "games")
public class Game {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "room_id")
    private GameRoom room;

    @Column(name = "selected_numbers", nullable = false, length = 50)
    private String selectedNumbers; // "3,15,22,45,67"

    @Column(name = "drawn_numbers", nullable = false, length = 100)
    private String drawnNumbers; // 20 números separados por coma
```

```

@Column(nullable = false)
private Integer hits;

@Column(name = "bet_amount", nullable = false, precision = 10, scale = 2)
private BigDecimal betAmount;

@Column(name = "win_amount", nullable = false, precision = 10, scale = 2)
private BigDecimal winAmount;
}

```

Relaciones JPA:

- `@ManyToOne` → Muchos juegos pertenecen a UN usuario. Crea una FK `user_id` en la tabla `games`
- `FetchType.LAZY` → No carga el usuario completo hasta que lo necesites (optimización de rendimiento)
- `@JoinColumn(name = "user_id")` → Nombre de la columna FK en la tabla

Si el profesor pregunta "¿Cómo agrego un nuevo campo a la entidad?": → Solo agrega el campo con sus anotaciones. Con `ddl-auto=update`, Hibernate agrega la columna automáticamente sin perder datos.

Entidades `GameRoom`, `GameRoomPlayer`, `ChatMessage`

Siguen el mismo patrón. `GameRoom` es la sala de juego multijugador, `GameRoomPlayer` es la tabla intermedia que relaciona jugadores con salas (con datos adicionales como apuesta, aciertos), y `ChatMessage` almacena mensajes del chat.

1.5 Repositorios — Acceso a datos

Spring Data JPA genera automáticamente las queries SQL. Solo defines una interfaz y Spring implementa los métodos.

`UserRepository.java`

```

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    Optional<User> findByEmail(String email);
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
    Optional<User> findByUsernameOrEmail(String username, String email);
}

```

¿Cómo funciona la magia? Spring analiza el nombre del método y genera la query SQL automáticamente:

- `findByUsername` → `SELECT * FROM users WHERE username = ?`
- `existsByEmail` → `SELECT COUNT(*) > 0 FROM users WHERE email = ?`
- `findByUsernameOrEmail` → `SELECT * FROM users WHERE username = ? OR email = ?`

JpaRepository hereda:

- `save(entity)` → INSERT o UPDATE
- `findById(id)` → SELECT por ID
- `findAll()` → SELECT *
- `deleteById(id)` → DELETE por ID
- `count()` → COUNT(*)

GameRepository.java — Queries personalizadas con @Query

```
@Query("SELECT COALESCE(SUM(g.betAmount), 0) FROM Game g WHERE g.user = :user")
BigDecimal sumBetAmountByUser(@Param("user") User user);
```

Esto es **JPQL** (Java Persistence Query Language), similar a SQL pero usando nombres de entidades Java en vez de tablas.

Si el profesor pregunta "¿Cómo busco por dos campos?": → `Optional<User>`

```
findByUsernameAndEmail(String username, String email); Spring genera: WHERE username = ?
AND email = ?
```

1.6 DTOs — Data Transfer Objects

Los DTOs son clases simples que definen QUÉ datos se envían/reciben en la API. **Regla de oro: nunca expongas entidades directamente.**

RegisterRequest.java — Validaciones en el DTO

```
@Data
public class RegisterRequest {
    @NotBlank(message = "El username es obligatorio")
    @Size(min = 3, max = 50)
    @Pattern(regexp = "^[a-zA-Z0-9_]+$", message = "Solo letras, números y guión bajo")
    private String username;

    @NotBlank @Email
    private String email;

    @NotBlank
    @Size(min = 8, message = "Mínimo 8 caracteres")
    @Pattern(regexp = "^(?=.*[A-Z])(?=.*\\d).+$", message = "Al menos una mayúscula y un número")
    private String password;

    @NotBlank
    private String confirmPassword;
}
```

Anotaciones de validación:

- `@NotBlank` → No puede ser null, vacío, ni solo espacios
- `@Size(min=3, max=50)` → Longitud entre 3 y 50 caracteres
- `@Pattern(regexp="...")` → Debe cumplir la expresión regular
- `@Email` → Formato de email válido
- `@Positive` → Debe ser mayor a 0
- `@DecimalMin("10")` → Mínimo 10

Estas validaciones se activan cuando el Controller usa `@Valid`:

```
public ResponseEntity<UserResponse> register(@Valid @RequestBody RegisterRequest request)
```

Si falla la validación, `GlobalExceptionHandler` captura el `MethodArgumentNotValidException` y devuelve un JSON con los errores.

1.7 Controladores — Endpoints REST

Los Controllers reciben peticiones HTTP y devuelven respuestas JSON. Son la capa más externa del backend.

`AuthController.java`

```
@Slf4j
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<UserResponse> register(@Valid @RequestBody
RegisterRequest request) {
        UserResponse response = authService.register(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @PostMapping("/login")
    public ResponseEntity<Map<String, Object>> login(@Valid @RequestBody
LoginRequest request) {
        Map<String, Object> response = authService.login(request);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/me")
    public ResponseEntity<UserResponse> getMe(@AuthenticationPrincipal UserDetails
userDetails) {
        UserResponse response = authService.getMe(userDetails.getUsername());
    }
}
```

```
        return ResponseEntity.ok(response);
    }
}
```

Anotaciones de Controller explicadas:

Anotación	Qué hace
<code>@RestController</code>	Combina <code>@Controller</code> + <code>@ResponseBody</code> . Todas las respuestas son JSON automáticamente
<code>@RequestMapping("/api/auth")</code>	Prefijo para todos los endpoints de esta clase
<code>@PostMapping("/register")</code>	Responde a <code>POST /api/auth/register</code>
<code>@GetMapping("/me")</code>	Responde a <code>GET /api/auth/me</code>
<code>@Valid</code>	Activa la validación Bean Validation del DTO
<code>@RequestBody</code>	Convierte el JSON del body de la petición al DTO Java automáticamente
<code>@AuthenticationPrincipal UserDetails</code>	Injecta los datos del usuario autenticado (extraídos del JWT)
<code>@PathVariable Long id</code>	Extrae el <code>{id}</code> de la URL. Ej: <code>/api/game/history/5</code> → <code>id = 5</code>
<code>@RequiredArgsConstructor</code>	Lombok genera el constructor con los campos <code>final</code> (inyección de dependencias)

ResponseEntity — Códigos HTTP:

- `ResponseEntity.ok(body)` → 200 OK
- `ResponseEntity.status(HttpStatus.CREATED).body(body)` → 201 Created
- `ResponseEntity.badRequest().body(body)` → 400 Bad Request
- `ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(body)` → 401

Si el profesor pregunta "¿Cómo agrego un endpoint para actualizar perfil?":

```
@PutMapping("/profile")
public ResponseEntity<UserResponse> updateProfile(
    @AuthenticationPrincipal UserDetails userDetails,
    @Valid @RequestBody UpdateProfileRequest request) {
    UserResponse response = userService.updateProfile(
        userDetails.getUsername(), request);
    return ResponseEntity.ok(response);
}
```

1.8 Servicios — Lógica de negocio

Los Services contienen TODA la lógica de negocio. Los Controllers solo reciben y responden; los Services procesan.

AuthService.java — Registro y Login

```
@Transactional
public UserResponse register(RegisterRequest request) {
    // 1. Verificar contraseñas iguales
    if (!request.getPassword().equals(request.getConfirmPassword()))
        throw new IllegalArgumentException("Las contraseñas no coinciden");

    // 2. Verificar username único
    if (userRepository.existsByUsername(request.getUsername()))
        throw new UserAlreadyExistsException("Username ya en uso");

    // 3. Verificar email único
    if (userRepository.existsByEmail(request.getEmail()))
        throw new UserAlreadyExistsException("Email ya registrado");

    // 4. Crear usuario con contraseña hasheada
    User user = User.builder()
        .username(request.getUsername())
        .email(request.getEmail())
        .password(passwordEncoder.encode(request.getPassword())) // BCrypt
        .build();

    User saved = userRepository.save(user);
    return mapToUserResponse(saved);
}
```

Conceptos clave:

- `@Transactional` → Si algo falla, la BD revierte todos los cambios (rollback). Garantiza consistencia
- `passwordEncoder.encode()` → BCrypt hashea la contraseña. Nunca se guarda en texto plano
- `mapToUserResponse()` → Convierte Entity a DTO. El password NUNCA se incluye en la respuesta

KenoLogicService.java — Lógica del juego

```
// Tabla de pagos bidimensional
private static final int[][] PAYTABLE = {
    {}, // 0 seleccionados
    {0, 3}, // 1 seleccionado: 1 acierto = 3x
    {0, 0, 9}, // 2 seleccionados: 2 aciertos = 9x
    // ... hasta 10 seleccionados
};

// Sorteo seguro con Fisher-Yates + SecureRandom
public List<Integer> generateDraw() {
    List<Integer> pool = IntStream.rangeClosed(1,
80).boxed().collect(Collectors.toList());
```

```

    for (int i = pool.size() - 1; i > 0; i--) {
        int j = secureRandom.nextInt(i + 1);
        Collections.swap(pool, i, j);
    }
    return pool.subList(0, 20).stream().sorted().collect(Collectors.toList());
}

```

¿Por qué SecureRandom y no Random? → `SecureRandom` usa algoritmos criptográficos (FIPS 140-2). `Random` es predecible. En un juego de azar es obligatorio usar `SecureRandom`.

¿Qué es Fisher-Yates shuffle? → Algoritmo que baraja una lista de forma uniforme (cada permutación tiene la misma probabilidad). Recorre la lista de atrás hacia adelante intercambiando cada elemento con uno aleatorio anterior.

1.9 Spring Security — JWT

`SecurityConfig.java` — Configuración de seguridad

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(AbstractHttpConfigurer::disable) // JWT no necesita
            CSRF
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/", "/index.html",
                    "/login.html", "/register.html", "/css/**", "/js/**",
                    "/game.html", "/history.html", "/profile.html",
                    "/ws/**", "/api/rooms/**").permitAll() // Rutas públicas
                .anyRequest().authenticated() // Todo lo demás
            requiere JWT
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // No
            usa sesiones
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class); // Filtro JWT
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Flujo de autenticación paso a paso:

1. Usuario hace POST `/api/auth/login` con username y contraseña
2. `AuthService.login()` verifica con `AuthenticationManager` → `CustomUserDetailsService` busca el usuario en BD → `BCryptPasswordEncoder` compara el hash
3. Si es válido, `JwtTokenProvider.generateToken()` crea un JWT firmado con la clave secreta
4. Se devuelve el token al frontend: `{"token": "eyJ...", "user": {...}}`
5. En cada petición posterior, el frontend envía el header: `Authorization: Bearer eyJ...`
6. `JwtAuthenticationFilter.doFilterInternal()` intercepta CADA petición:
 - Extrae el token del header
 - Valida la firma y la expiración
 - Carga el usuario desde la BD
 - Establece la autenticación en el `SecurityContext`
7. Spring Security verifica si el usuario tiene acceso al endpoint

JwtTokenProvider.java

```
public String buildToken(String username) {
    return Jwts.builder()
        .setSubject(username)           // ¿Quién es?
        .setIssuedAt(new Date())        // ¿Cuándo se creó?
        .setExpiration(new Date(now + jwtExpiration)) // ¿Cuándo expira?
        .signWith(getSigningKey(), SignatureAlgorithm.HS256) // Firma con
HMAC-SHA256
        .compact();
}
```

JwtAuthenticationFilter.java

```
@Override
protected void doFilterInternal(HttpServletRequest request, ...) {
    String token = extractTokenFromRequest(request); // Saca el "Bearer xxx"

    if (StringUtils.hasText(token) && jwtTokenProvider.validateToken(token)) {
        String username = jwtTokenProvider.getUsernameFromToken(token);
        UserDetails userDetails = userDetailsService.loadUserByUsername(username);

        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());

        SecurityContextHolder.getContext().setAuthentication(authentication);
    }

    filterChain.doFilter(request, response); // Continúa al siguiente filtro
}
```

Si el profesor pregunta "¿Cómo protejo un endpoint nuevo?": → Si no está en la lista de `permitAll()`, ya está protegido automáticamente. Solo los endpoints públicos necesitan declararse.

1.10 Manejo de excepciones — `GlobalExceptionHandler.java`

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserAlreadyExistsException.class)
    public ResponseEntity<Map<String, Object>>
handleUserAlreadyExists(UserAlreadyExistsException ex) {
        return buildResponse(HttpStatus.CONFLICT, ex.getMessage()); // 409
    }

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<Map<String, Object>>
handleBadCredentials(BadCredentialsException ex) {
        return buildResponse(HttpStatus.UNAUTHORIZED, "Usuario o contraseña
incorrectos"); // 401
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>>
handleValidationErrors(MethodArgumentNotValidException ex) {
        Map<String, String> errores = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach(error -> {
            String campo = ((FieldError) error).getField();
            errores.put(campo, error.getDefaultMessage());
        });
        // Responde con: { "errores": { "email": "formato inválido", "password":
"mínimo 8 caracteres" } }
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGenericException(Exception
ex) {
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR, "Error interno");
        // 500, NUNCA expone stack trace
    }
}
```

`@RestControllerAdvice` captura TODAS las excepciones de TODOS los Controllers en un solo lugar. Así no necesitas try-catch en cada endpoint.

1.11 WebSocket + STOMP — Tiempo real

`WebSocketConfig.java`

```

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic"); // Servidor → Cliente
        config.setApplicationDestinationPrefixes("/app"); // Cliente → Servidor
    }
    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOriginPatterns("*");
    }
}

```

Cómo funciona:

- El frontend se conecta a `ws://localhost:8080/ws`
- Se suscribe a canales como `/topic/room/1/draw` para recibir números del sorteo
- Envía mensajes a `/app/room.play` para hacer jugadas
- `SimpMessagingTemplate.convertAndSend()` en el backend envía datos a todos los suscriptores

RoomScheduler.java — Timer automático

```

@Scheduled(fixedRate = 1000) // Se ejecuta cada 1 segundo
public void tickTimers() { ... }

@Scheduled(fixedRate = 30000) // Cada 30 segundos
public void ensureAvailableRoom() { ... }

```

`@EnableScheduling` + `@Scheduled` ejecuta métodos periódicamente sin intervención del usuario.

1.12 CORS — CorsConfig.java

```

@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
        .allowedOriginPatterns(allowedOrigins)
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("*")
        .allowCredentials(true)
        .maxAge(3600);
}

```

¿Qué es CORS? → Cross-Origin Resource Sharing. El navegador bloquea peticiones de un dominio a otro por seguridad. CORS le dice al navegador "este origen sí puede acceder a mi API".

1.13 Resumen de endpoints del proyecto

Método	Endpoint	Controller	Público	Qué hace
POST	/api/auth/register	AuthController	✓	Registrar usuario
POST	/api/auth/login	AuthController	✓	Login, devuelve JWT
POST	/api/auth/logout	AuthController	✓	Logout (simbólico)
GET	/api/auth/me	AuthController	✗	Datos del usuario autenticado
GET	/api/user/profile	UserController	✗	Perfil del usuario
GET	/api/user/balance	UserController	✗	Saldo del usuario
POST	/api/user/recharge	UserController	✗	Recargar créditos
GET	/api/game/history	GameController	✗	Historial de partidas
GET	/api/game/history/{id}	GameController	✗	Detalle de una partida
GET	/api/game/stats	GameController	✗	Estadísticas del jugador
GET	/api/rooms	GameRoomController	✓	Salas disponibles
POST	/api/rooms	GameRoomController	✓	Crear/obtener sala
POST	/api/rooms/{id}/join	GameRoomController	✗	Unirse a una sala

PARTE 2 — HTML (Frontend)

2.1 Estructura de páginas

El proyecto tiene 6 páginas HTML, todas usan HTML5 semántico y son servidas como archivos estáticos por Spring Boot desde `src/main/resources/static/`.

Página	Ruta	Requiere auth	Descripción
index.html	/	No	Landing page
login.html	/login.html	No	Formulario de login
register.html	/register.html	No	Formulario de registro
game.html	/game.html	Sí (JS)	Tablero de juego + salas
history.html	/history.html	Sí (JS)	Historial de partidas
profile.html	/profile.html	Sí (JS)	Perfil del usuario

2.2 login.html — Formulario de autenticación

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Velour Keno – Iniciar Sesión</title>
  <link rel="stylesheet" href="/css/styles.css">
  <link rel="stylesheet" href="/css/auth.css">
</head>
```

Etiquetas `<head>` explicadas:

- `<!DOCTYPE html>` → Declara HTML5
- `<html lang="es">` → Idioma español (accesibilidad, SEO)
- `<meta charset="UTF-8">` → Soporte para tildes y ñ
- `<meta name="viewport" ...>` → **Esencial para responsive**. Sin esto, en móvil la página se ve diminuta
- `<link rel="stylesheet">` → Carga CSS externo. Se cargan `styles.css` (base) y `auth.css` (específico de auth)

Formulario:

```
<form id="loginForm" novalidate>
  <div class="form-group">
    <label for="usernameOrEmail">Usuario o Email</label>
    <input type="text" id="usernameOrEmail" name="usernameOrEmail"
      placeholder="usuario o email@ejemplo.com"
      autocomplete="username" required />
    <span class="field-error" id="usernameOrEmailError"></span>
  </div>

  <div class="form-group">
    <label for="password">Contraseña</label>
    <div class="input-password-wrapper">
      <input type="password" id="password" name="password"
        placeholder="....."
        autocomplete="current-password" required />
      <button type="button" class="toggle-password"
        aria-label="Mostrar contraseña"
        onclick="togglePassword('password')">👁️</button>
    </div>
    <span class="field-error" id="passwordError"></span>
  </div>

  <button type="submit" class="btn-primary" id="loginBtn">
    <span class="btn-text">Iniciar Sesión</span>
    <span class="btn-loader" style="display:none">Cargando...</span>
  </button>
</form>
```

Conceptos HTML que el profesor puede preguntar:

Atributo/Elemento	Qué hace	Por qué importa
<code>novalidate</code>	Desactiva la validación nativa del navegador	Usamos validación personalizada con JS
<code>for="usernameOrEmail"</code>	Vincula el <code><label></code> con el <code><input></code> por su <code>id</code>	Accesibilidad: al clicar el label, enfoca el input
<code>type="password"</code>	Oculto los caracteres del input	Seguridad visual
<code>autocomplete="username"</code>	El navegador sugiere autocompletado	Mejora UX
<code>required</code>	Atributo HTML5 de validación (desactivado por <code>novalidate</code>)	Se usa como señal semántica
<code>placeholder</code>	Texto de ejemplo gris dentro del input	Desaparece al escribir
<code>aria-label</code>	Texto invisible para lectores de pantalla	Accesibilidad para el botón de ojo
<code>type="button"</code>	El botón NO envía el formulario	Sin esto, al clicar "👁" se enviaría el form
<code></code>	Contenedor para mensajes de error por campo	Se llena dinámicamente con JS

Si el profesor dice "Hazme un formulario de login": → Necesitas: `<form>`, `<label>` + `<input>` para cada campo, `<button type="submit">`, y vincular con JS para hacer el `fetch` al backend.

2.3 `register.html` — Registro con indicador de fuerza

Agrega dos cosas extra respecto al login:

```
<!-- Indicador de fuerza de contraseña -->
<div class="password-strength" id="passwordStrength">
  <div class="strength-bar">
    <div class="strength-fill" id="strengthFill"></div>
  </div>
  <span class="strength-label" id="strengthLabel"></span>
</div>
```

Esto se anima con JS (`auth.js`) que evalúa: longitud ≥ 8 , mayúscula, número, carácter especial.

2.4 `index.html` — Landing page

Estructura semántica completa:

```
<nav class="landing-navbar">...</nav>           <!-- Navegación -->
<section class="hero" role="banner">...</section> <!-- Hero principal -->
```

```
<section class="features" id="features">...</section> <!-- Características -->
<footer class="landing-footer" role="contentinfo">...</footer>
```

Etiquetas semánticas HTML5:

- `<nav>` → Sección de navegación
- `<section>` → Sección temática del contenido
- `<article>` → Contenido independiente (las feature cards)
- `<main>` → Contenido principal de la página
- `<aside>` → Contenido relacionado pero secundario (panel de jugadores)
- `<footer>` → Pie de página
- `role="banner"` / `role="contentinfo"` → Roles ARIA para accesibilidad

Logo SVG inline:

```
<svg width="36" height="36" viewBox="0 0 100 100" fill="none">
  <polygon points="50,10 84.6,30 84.6,70 50,90 15.4,70 15.4,30"
    fill="none" stroke="#6B0F1A" stroke-width="4"/>
  <polygon points="50,22 74.2,36 74.2,64 50,78 25.8,64 25.8,36"
    fill="none" stroke="#C9A84C" stroke-width="3"/>
  <circle cx="50" cy="50" r="8" fill="#6B0F1A"/>
  <circle cx="50" cy="50" r="3" fill="#C9A84C"/>
</svg>
```

Dos hexágonos concéntricos (rojo vino + dorado) con un punto central. Es SVG inline para no depender de archivos de imagen.

2.5 `game.html` — La página más compleja

Tiene 5 secciones principales:

1. **Lobby** (`#lobbyView`) — Grid de tarjetas de salas disponibles
2. **Game View** (`#gameView`) — Layout de 3 columnas: jugadores | tablero | controles
3. **Chat** (`#chatPanel`) — Panel fijo abajo a la derecha, colapsable
4. **Overlay de resultados** (`#resultsOverlay`) — Modal con resultados de cada ronda
5. **Modales** — Paytable, estadísticas, cómo jugar

Tablero de juego:

```
<div id="gameBoard" class="game-board" role="grid"
  aria-label="Tablero de números 1 al 80">
  <!-- 80 botones inyectados por renderBoard() en game.js -->
</div>
```

Los 80 números se crean dinámicamente con JavaScript, no están escritos en el HTML. Esto es más eficiente y mantenible.

Atributos ARIA para accesibilidad:

```
<span id="timerDisplay" aria-live="polite">3:30</span>
<!-- aria-live="polite" avisa a lectores de pantalla cuando cambia el contenido -->

<label for="betAmount" class="sr-only">Cantidad de apuesta</label>
<!-- sr-only: visible solo para lectores de pantalla -->
```

PARTE 3 — CSS (Estilos)

3.1 `styles.css` — Variables y sistema de diseño base

```
:root {
  --color-bg:           #0A0A0A;      /* Fondo principal – casi negro */
  --color-bg-card:      #111111;      /* Fondo de tarjetas */
  --color-bg-input:      #1A1A1A;      /* Fondo de inputs */
  --color-primary:       #6B0F1A;      /* Rojo vino – botones principales */
  --color-primary-hover: #A01020;      /* Rojo más claro al hover */
  --color-gold:          #C9A84C;      /* Dorado – acentos, saldo, premios */
  --color-gold-hover:    #E2C06A;      /* Dorado más claro */
  --color-text:          #F5F0EB;      /* Texto principal – blanco cálido */
  --color-text-muted:    #8A7A75;      /* Texto secundario – gris cálido */
  --color-error:         #E53E3E;      /* Rojo – errores */
  --color-success:       #38A169;      /* Verde – éxito, aciertos */
  --color-border:        #2A2A2A;      /* Bordes sutiles */
  --radius:              12px;
  --transition:          0.2s ease;
}
```

¿Qué son las CSS Custom Properties (variables)? → Se definen en `:root` (elemento raíz = `<html>`) y se usan con `var(--nombre)`. Permiten cambiar el tema completo cambiando una sola línea.

Si el profesor dice "Hazme esta ventana azul": → Cambias `--color-primary: #2563EB;` y `--color-primary-hover: #1D4ED8;` en `:root`. TODOS los botones, fondos y acentos cambian instantáneamente.

Reset CSS:

```
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
```

- `box-sizing: border-box` → El padding y border se incluyen dentro del width/height (no se suman). Esencial para layouts predecibles
- `margin: 0; padding: 0` → Elimina los estilos por defecto del navegador

Tipografía:

```
body {  
  font-family: 'Georgia', serif; /* Títulos: Georgia (serif elegante) */  
  background-color: var(--color-bg);  
  color: var(--color-text);  
}
```

- Georgia (serif) → Para títulos y branding — da sensación premium
- Arial (sans-serif) → Para texto UI, labels, datos — legible y funcional

3.2 Botón primario — Anatomía completa

```
.btn-primary {  
  width: 100%;  
  padding: 14px;  
  background: var(--color-primary);  
  color: var(--color-text);  
  border: none;  
  border-radius: var(--radius); /* 12px esquinas redondeadas */  
  font-size: 15px;  
  font-weight: 600;  
  letter-spacing: 0.05em; /* Espaciado entre letras */  
  text-transform: uppercase; /* MAYÚSCULAS */  
  transition: background var(--transition), transform var(--transition);  
  min-height: 48px; /* Mínimo 48px para accesibilidad táctil */  
}
```

```
.btn-primary:hover { background: var(--color-primary-hover); }  
.btn-primary:active { transform: scale(0.98); } /* Efecto de presión */  
.btn-primary:disabled { opacity: 0.6; cursor: not-allowed; }
```

Conceptos CSS clave:

- `transition` → Animación suave entre estados (0.2s al pasar el mouse)
- `:hover` → Cuando el mouse está encima
- `:active` → Cuando se está clickeando
- `:disabled` → Cuando el botón está deshabilitado
- `min-height: 48px` → Google recomienda mínimo 48px para elementos táctiles en móvil

3.3 `auth.css` — Layout centrado con gradiente

```
.auth-container {  
  min-height: 100vh; /* Ocupa toda la pantalla */  
  display: flex;  
  align-items: center; /* Centra verticalmente */  
}
```

```
justify-content: center;          /* Centra horizontalmente */
background:
  radial-gradient(ellipse at 20% 50%, rgba(107, 15, 26, 0.15) 0%,
transparent 60%),
  radial-gradient(ellipse at 80% 50%, rgba(201, 168, 76, 0.05) 0%,
transparent 60%),
  var(--color-bg);
}
```

Flexbox para centrar — la forma más común:

```
display: flex;
align-items: center;
justify-content: center;
```

Gradientes radiales superpuestos:

- Primer gradiente: brillo rojo vino suave a la izquierda (20% horizontal)
- Segundo gradiente: brillo dorado sutil a la derecha (80% horizontal)
- Base: color de fondo negro

Si el profesor pregunta "¿Cómo centro un div?": → Padre con `display: flex; align-items: center; justify-content: center;`

3.4 game.css — Grid layout complejo

Layout de 3 columnas

```
.game-view {
  display: grid;
  grid-template-columns: 200px 1fr 340px; /* Fijo | Flexible | Fijo */
  gap: 20px;
  padding: 20px 40px;
  min-height: calc(100vh - 72px); /* Toda la pantalla menos el navbar */
}
```

CSS Grid explicado:

- `200px` → Panel de jugadores (ancho fijo)
- `1fr` → Tablero de juego (ocupa todo el espacio restante)
- `340px` → Panel de controles (ancho fijo)
- `gap: 20px` → Espacio entre las columnas
- `calc(100vh - 72px)` → 100% de la altura visible menos los 72px del navbar

Cuadrícula de números 10×8

```
.game-board {  
  display: grid;  
  grid-template-columns: repeat(10, 1fr); /* 10 columnas iguales */  
  gap: 8px;  
  user-select: none; /* No permite seleccionar texto al clicar */  
}  
  
.board-number {  
  aspect-ratio: 1; /* Cuadrado perfecto */  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  border-radius: 8px;  
  cursor: pointer;  
  transition: all 0.15s ease;  
  min-width: 44px;  
  min-height: 44px;  
}
```

Estados visuales del tablero

```
/* Default */  
.board-number { background: #1A1A1A; border-color: #2A2A2A; color: #8A7A75; }  
  
/* Seleccionado por el jugador */  
.board-number.selected {  
  background: #6B0F1A; border-color: #A01020; color: #F5F0EB;  
  transform: scale(1.05); /* Se agranda 5% */  
}  
  
/* Número sorteado */  
.board-number.drawn { border-color: rgba(201, 168, 76, 0.5); color: rgba(201, 168, 76, 0.7); }  
  
/* Acierto */  
.board-number.hit {  
  background: #0F1A0F; border-color: #38A169; color: #38A169;  
  box-shadow: 0 0 8px rgba(56, 161, 105, 0.3); /* Brillo verde */  
}  
  
/* Fallo */  
.board-number.miss { background: #1A0A0A; border-color: #E53E3E; color: #E53E3E; }
```

Cómo se aplican: JavaScript agrega/quita clases con `classList.add('hit')` y `classList.remove('selected')`.

Animación de sorteo

```
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.15); box-shadow: 0 0 12px rgba(201, 168, 76, 0.5); }
  100% { transform: scale(1); }
}

.board-number.drawing { animation: pulse 0.4s ease; }
```

`@keyframes` define los pasos de la animación. Se activa cuando JS agrega la clase `drawing`.

Animación de timer urgente

```
@keyframes blink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.5; }
}

.timer-display.urgent { color: #E53E3E; animation: blink 1s infinite; }
```

3.5 navbar.css — Navegación fija

```
.navbar {
  position: fixed;           /* Se queda fija al hacer scroll */
  top: 0; left: 0;
  width: 100%;
  height: 72px;
  z-index: 1000;           /* Siempre encima de todo */
  display: flex;
  align-items: center;
  justify-content: space-between; /* Logo | Links | Derecha */
}
```

Toast notifications:

```
.toast {
  position: fixed;
  bottom: 24px; right: 24px;
  z-index: 9999;
  opacity: 0;
  transform: translateY(8px);
  transition: opacity 0.3s ease, transform 0.3s ease;
}

.toast-visible {
  opacity: 1;
}
```

```
transform: none; /* Vuelve a su posición normal */
}
```

Aparece deslizándose desde abajo con fade-in, y desaparece con fade-out.

3.6 Responsive Design — Media Queries

```
@media (max-width: 1024px) {
  .game-view { grid-template-columns: 1fr; } /* Una sola columna */
  .players-panel { display: none; } /* Oculta panel de jugadores */
  .game-board { grid-template-columns: repeat(8, 1fr); } /* 8 columnas en vez
de 10 */
}

@media (max-width: 768px) {
  .board-number { font-size: 12px; min-width: 36px; min-height: 36px; }
  .navbar { padding: 0 16px; }
  .nav-links { display: none; } /* Oculta links en móvil */
  .chat-panel { width: 100%; right: 0; }
}

@media (max-width: 480px) {
  .auth-card { padding: 28px 20px; }
  .stats-grid { grid-template-columns: 1fr !important; } /* Una columna */
}
```

Breakpoints:

- 1024px → Tablets en landscape
- 768px → Tablets en portrait / móvil grande
- 480px → Móviles pequeños

Si el profesor dice "Haz que funcione en móvil": → Necesitas `<meta name="viewport">` en el HTML + media queries que cambien el grid a 1 columna y oculten elementos no esenciales.

PARTE 4 — JAVASCRIPT (Frontend)

4.1 api.js — Comunicación con el backend

```
const API_BASE = '/api';
const TOKEN_KEY = 'velour_token';

function getToken() { return localStorage.getItem(TOKEN_KEY); }
function saveToken(t) { localStorage.setItem(TOKEN_KEY, t); }
function removeToken() { localStorage.removeItem(TOKEN_KEY); }
function isAuthenticated() { return !!getToken(); }
```

```

async function apiFetch(endpoint, method = 'GET', body = null) {
  const headers = new Headers();
  headers.append('Content-Type', 'application/json');
  const token = getToken();
  if (token) headers.append('Authorization', `Bearer ${token}`); // JWT auto-
  inyectado

  const response = await fetch(`${API_BASE}${endpoint}`, {
    method, headers,
    body: body ? JSON.stringify(body) : undefined
  });

  if (response.status === 401 || response.status === 403) {
    removeToken();
    window.location.href = '/login.html'; // Sesión expirada → login
    return;
  }

  if (!response.ok) throw { status: response.status, data: await response.json() };
  return await response.json();
}

```

Conceptos clave:

- `localStorage` → Almacenamiento persistente del navegador. El JWT se guarda aquí
- `fetch()` → API nativa del navegador para hacer peticiones HTTP
- `async/await` → Sintaxis moderna para manejar promesas (código asíncrono que parece síncrono)
- El token se envía automáticamente en CADA petición protegida

4.2 `auth.js` — Login y registro

```

loginForm.addEventListener('submit', async (e) => {
  e.preventDefault(); // Evita que el form se envíe de forma tradicional
  clearErrors();

  // Validación frontend
  if (!usernameOrEmail) { showFieldError('usernameOrEmail', 'Obligatorio');
  return; }

  setLoading('loginBtn', true); // Muestra "Cargando..."

  const response = await fetch(`${API_BASE}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usernameOrEmail, password })
  });

  const data = await response.json();
  if (!response.ok) { showGlobalError(data.mensaje); return; }
}

```

```
    saveToken(data.token); // Guarda JWT
    window.location.href = '/game.html'; // Redirige al juego
  });
```

Patrón de validación frontend + backend:

1. Frontend valida primero (UX instantánea)
2. Si pasa, envía al backend
3. Backend valida de nuevo (seguridad, nunca confiar en el frontend)
4. Si falla en backend, muestra errores del servidor

4.3 `utils.js` — Funciones utilitarias

```
function formatCredits(amount) {
  if (amount == null || isNaN(amount)) return '0 ♦';
  return Number(amount).toLocaleString('es-CO') + ' ♦'; // 1.000 ♦
}

function showToast(message, type = 'info') {
  const toast = document.createElement('div');
  toast.className = `toast toast-${type}`;
  toast.textContent = message;
  document.body.appendChild(toast);
  requestAnimationFrame(() => toast.classList.add('toast-visible'));
  setTimeout(() => {
    toast.classList.remove('toast-visible');
    setTimeout(() => toast.remove(), 400);
  }, 3000);
}
```

4.4 `websocket.js` — Comunicación en tiempo real

```
function connectWebSocket() {
  var wsUrl = protocol + '//' + window.location.host + '/ws';
  stompClient = new StompJs.Client({
    brokerURL: wsUrl,
    connectHeaders: { Authorization: 'Bearer ' + getToken() },
    onConnect: onWebSocketConnected,
  });
  stompClient.activate();
}

function joinRoom(roomId) {
  // Suscribirse a 5 canales de la sala:
  stompClient.subscribe('/topic/room/' + roomId + '/players', ...); // Lista jugadores
  stompClient.subscribe('/topic/room/' + roomId + '/timer', ...); //
```

```

Temporizador
    stompClient.subscribe('/topic/room/' + roomId + '/draw', ...);    // Números
sorteados
    stompClient.subscribe('/topic/room/' + roomId + '/results', ...); //
Resultados
    stompClient.subscribe('/topic/room/' + roomId + '/chat', ...);    // Chat
}

function sendPlay(selectedNumbers, betAmount) {
    stompClient.publish({
        destination: '/app/room.play',
        body: JSON.stringify({ roomId, selectedNumbers, betAmount })
    });
}

```

4.5 game.js — Lógica del tablero

```

function renderBoard() {
    for (var i = 1; i <= 80; i++) {
        var btn = document.createElement('button');
        btn.className = 'board-number';
        btn.setAttribute('data-number', i);
        btn.textContent = i;
        btn.onclick = makeToggleHandler(i);
        container.appendChild(btn);
    }
}

function toggleNumber(number) {
    if (isDrawing) return; // No permite cambiar durante el sorteo
    if (selectedNumbers.length >= 10) { showToast('Máximo 10', 'warning'); return; }
    selectedNumbers.push(number);
    btn.classList.add('selected');
}

```

PARTE 5 — PREGUNTAS FRECUENTES DEL PROFESOR

"Hazme un login en Spring Boot"

Necesitas: Entity User, UserRepository, AuthService con BCrypt, AuthController con `/login`, SecurityConfig con JWT, JwtTokenProvider, JwtAuthenticationFilter, login.html con formulario, auth.js con fetch al endpoint.

"Hazme esta ventana azul"

Cambia `--color-primary: #2563EB` en `:root` de styles.css. Todos los botones y acentos cambian.

"Agrega un campo nuevo al formulario"

1. Agrega el campo en el DTO (RegisterRequest) con validaciones
2. Agrega la columna en la Entity (User) con `@Column`
3. Agrega el `<input>` en el HTML con `<label>`
4. Agrega la validación en JS (auth.js)

"Cambia el color de fondo de la página"

→ `--color-bg: #1a1a2e;` en styles.css

"¿Cómo protejo un endpoint?"

→ No lo pongas en `permitAll()` de SecurityConfig. Automáticamente requiere JWT.

"¿Cómo funciona el responsive?"

→ `<meta name="viewport">` + media queries `@media (max-width: 768px)` que cambian grid, ocultan paneles, reducen fuentes.

"¿Cómo se comunican frontend y backend?"

→ Frontend hace `fetch('/api/endpoint')` con el JWT en el header `Authorization: Bearer token`. Backend responde JSON.

"¿Qué pasa si la contraseña es incorrecta?"

→ AuthenticationManager lanza `BadCredentialsException` → `GlobalExceptionHandler` la captura → devuelve JSON con status 401 y mensaje "Usuario o contraseña incorrectos" → Frontend muestra el error.

INVENTARIO COMPLETO DE ARCHIVOS

Backend Java (22 archivos)

Capa	Archivo	Rol
Main	<code>KenoApplication.java</code>	Punto de entrada
Config	<code>SecurityConfig.java</code>	JWT + rutas protegidas
Config	<code>CorsConfig.java</code>	Permisos cross-origin
Config	<code>WebSocketConfig.java</code>	STOMP endpoints
Entity	<code>User.java</code>	Tabla users
Entity	<code>Game.java</code>	Tabla games
Entity	<code>GameRoom.java</code>	Tabla game_rooms
Entity	<code>GameRoomPlayer.java</code>	Tabla game_room_players

Capa	Archivo	Rol
Entity	ChatMessage.java	Tabla chat_messages
Repository	UserRepository.java	Queries de usuarios
Repository	GameRepository.java	Queries de partidas
Repository	GameRoomRepository.java	Queries de salas
Repository	GameRoomPlayerRepository.java	Queries de jugadores en salas
Repository	ChatMessageRepository.java	Queries de chat
DTO	LoginRequest, RegisterRequest, RechargeRequest, RoomPlayRequest, RoomChatRequest, RoomJoinRequest, UserResponse, GameResultResponse, GameRoomResponse	Transferencia de datos
Controller	AuthController.java	Endpoints de auth
Controller	UserController.java	Endpoints de usuario
Controller	GameController.java	Endpoints de historial/stats
Controller	GameRoomController.java	Endpoints + WebSocket de salas
Service	AuthService.java	Lógica de registro/login
Service	UserService.java	Lógica de perfil/recarga
Service	KenoLogicService.java	Lógica del juego Keno
Service	GameRoomService.java	Lógica de salas multiplayer
Service	ChatService.java	Lógica de chat
Security	JwtTokenProvider.java	Genera/valida JWT
Security	JwtAuthenticationFilter.java	Filtro HTTP para JWT
Security	CustomUserDetailsService.java	Carga usuario para Spring Security
Exception	GlobalExceptionHandler.java	Manejo centralizado de errores
Exception	3 excepciones custom	UserAlreadyExists, InsufficientBalance, ResourceNotFound
Scheduler	RoomScheduler.java	Timer automático de salas

Frontend (14 archivos)

Tipo	Archivo	Líneas
HTML	<code>index.html</code>	Landing page
HTML	<code>login.html</code>	Formulario login
HTML	<code>register.html</code>	Formulario registro
HTML	<code>game.html</code>	Tablero + salas + chat + modales
HTML	<code>history.html</code>	Historial paginado + stats
HTML	<code>profile.html</code>	Perfil + cambio contraseña
CSS	<code>styles.css</code>	203 líneas — Variables, reset, forms, botones
CSS	<code>auth.css</code>	68 líneas — Layout de login/registro
CSS	<code>navbar.css</code>	236 líneas — Navbar, toasts, profile menu
CSS	<code>game.css</code>	973 líneas — Tablero, salas, chat, responsive
JS	<code>api.js</code>	171 líneas — Fetch + JWT
JS	<code>auth.js</code>	248 líneas — Login + registro + validación
JS	<code>utils.js</code>	171 líneas — Format, toast, helpers
JS	<code>navbar.js</code>	194 líneas — Navbar, recarga, profile menu
JS	<code>game.js</code>	638 líneas — Tablero, apuestas, animaciones
JS	<code>websocket.js</code>	238 líneas — STOMP, salas, chat